

Intro to Mininet

Introduction:

This lab introduces Mininet, which is a virtual testbed used for testing network tools and protocols.

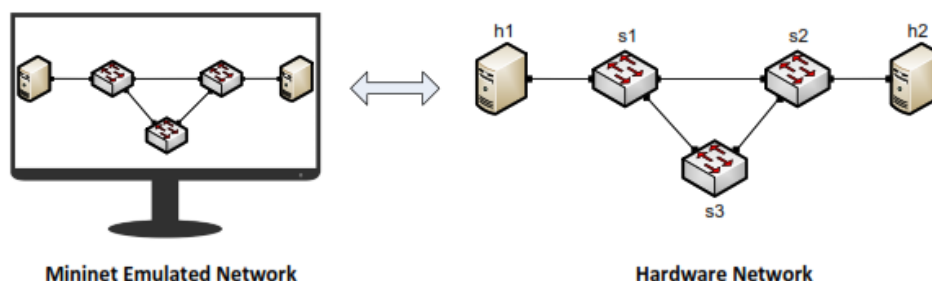
Objectives:

- Understand mininet and its uses for testing network topologies
- Installation of Mininet and importing Mininet VM on VMware/VirtualBox
- Starting mininet from CLI (GUI we will cover in other labs)
- Construct a simple topology
- Saving and loading topologies
- Running commands

Mininet

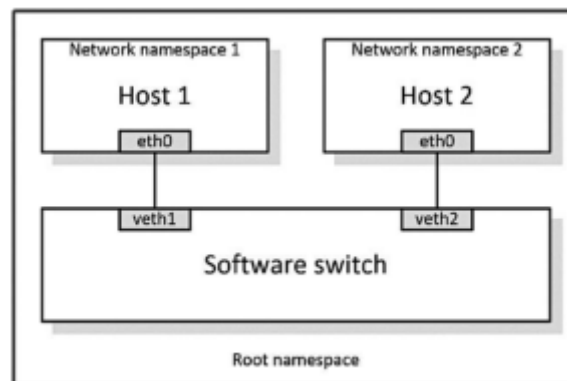
Mininet is a virtual testbed enabling the development and testing of network tools and protocols. With a single command, Mininet can create a realistic virtual network on any type of machine (Virtual Machine (VM), cloud-hosted, or native). Therefore, it provides an inexpensive solution and streamlined development running in line with production networks. Mininet offers the following features:

- Fast prototyping for new networking protocols.
- Simplified testing for complex topologies without the need of buying expensive hardware.
- Realistic execution as it runs real code on the Unix and Linux kernels.
- Open-source environment backed by a large community contributing extensive documentation.



Mininet is useful for development, teaching, and research as it is easy to customize and interact with it through the CLI or the GUI. Mininet was originally designed to experiment with OpenFlow and Software-Defined Networking (SDN). This lab, however, only focuses on emulating a simple network environment without SDN-based devices.

Mininet's logical nodes can be connected into networks. These nodes are sometimes called containers, or more accurately, network namespaces. Containers consume sufficiently fewer resources than networks of over a thousand nodes have created, running on a single laptop. A Mininet container is a process (or group of processes) that no longer has access to all the host system's native network interfaces. Containers are then assigned virtual Ethernet interfaces, which are connected to other containers through a virtual switch. Mininet connects a host and a switch using a virtual Ethernet (veth) link. The veth link is analogous to a wire connecting two virtual interfaces, as illustrated below.



Each container is an independent network namespace, a lightweight virtualization feature that provides individual processes with separate network interfaces, routing tables, and Address Resolution Protocol (ARP) tables.

Mininet provides network emulation opposed to simulation, allowing all network software at any layer to be simply run as is, i.e., nodes run the native network software of the physical machine. On the other hand, in a simulated environment applications and protocol implementations need to be ported to run within the simulator before they can be used.

Lab settings

The username and password for using Mininet machine are

If using Mininet VM, the username and password is 'mininet'.

MININET installation

Download Link: <http://mininet.org/download/>

There are couple of options to install the Mininet.

- 1 - Installation from Packages
- 2 - Native installation from Source files ([LINK](#))
- 3 - Mininet installation using Virtual Machine ([LINK](#))

The easiest way is to download the Mininet VM from the website link above. However, if you want to use the GUI of Mininet which is Miniedit, then you need to install via source files on a separate Ubuntu VM or SSH into the pre-built Mininet VM machine. We will look at both options for installation.

Mininet Installation using Packages.

Use the following command to install the mininet.

```
sudo apt-get install mininet
```

Once installed, use the following command to test the mininet.

```
sudo mn --switch ovsbr --test pingall
```

Install OpenVSwitch, using the following command.

```
sudo apt-get install openvswitch-switch
```

```
sudo service openvswitch-switch start
```

After that type

```
sudo mn -c
```

And then,

```
sudo mn
```

It will take you to the mininet console which you can use to test your network topology.

We will discuss further about this default topology at the end of this lab manual.

Only, if you need to uninstall it.

```
sudo apt-get uninstall mininet    # if you have installed a Mininet apt package
```

Mininet Installation using Sources.

It can be installed simply by fetching the source files from git repository and using install.sh command with switches.

It is strongly recommended to use more recent Ubuntu or Debian releases, because they include newer versions of Open vSwitch. (Fedora also includes recent OvS releases.)

Download link for Ubuntu ISO LTS version. You first need to install the Linux OS (ubuntu) to install mininet on it.

<https://ubuntu.com/download/desktop>

To install natively from source, first you need to get the source code. Go to the folder where you want to download and keep the files and then run the following command. These source files are also available on the mininet website (<https://github.com/mininet/mininet/releases/>). You can also download it from their and place it in a folder where you want to install and keep the files. You can also use the following command.

```
git clone https://github.com/mininet/mininet
```

Note that the above git command will check out the latest and greatest Mininet (which we recommend!) If you want to run the last tagged/released version of Mininet - or any other version - you may check that version out explicitly:

```
cd mininet
```

```
git tag # list available versions
```

```
git checkout -b mininet-2.3.0 2.3.0 # or whatever version you wish to install
```

```
cd ..
```

Once you have the source tree, goto the folder. The command to install Mininet is

```
mininet/util/install.sh [options]
```

Use as following for the installation.

```
mininet/util/install.sh -a
```

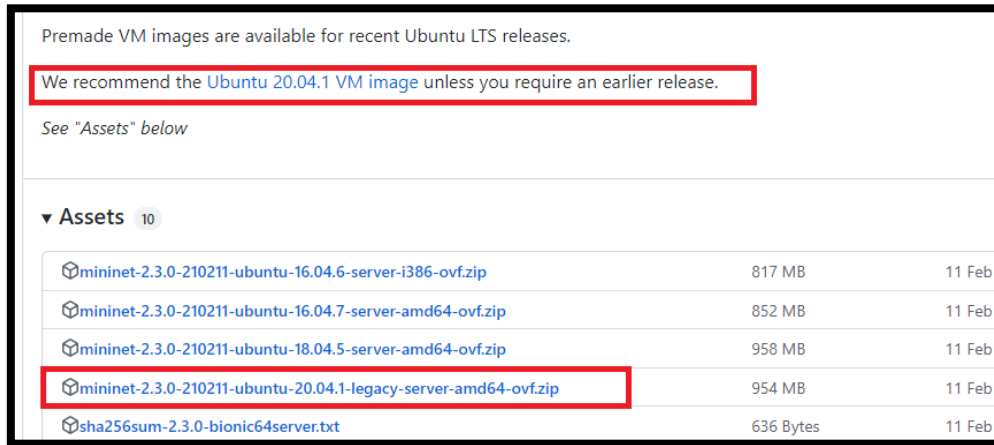
Typical install.sh options include:

- -a: install everything that is included in the Mininet VM, including dependencies like Open vSwitch as well the additions like the OpenFlow wireshark dissector and POX. By default these tools will be built in directories created in your home directory.
- -nfv: install Mininet, the OpenFlow reference switch, and Open vSwitch
- -s mydir: use this option before other options to place source/build trees in a specified directory rather than in your home directory.

Mininet VM download:

We will use that Mininet VM for all our practical. So, its installation process is given here.

1. Download a Mininet VM Image from Mininet Releases ([LINK](#))



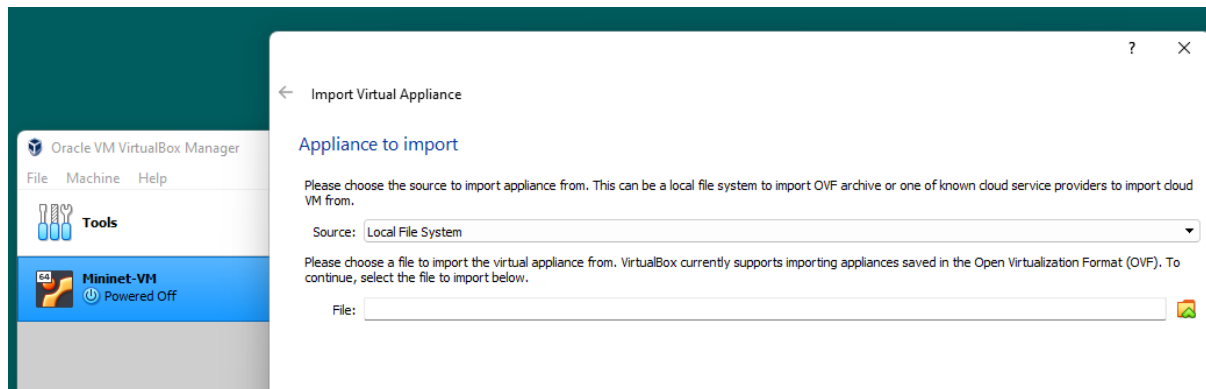
2. Download and install a hypervisor (virtualization system) like VMware WS, VirtualBox, VMWare Fusion, Qemu, hyper-V, or KVM, according to your system. ** Its import on VirtualBox is also very simple.
3. Import or add the Mininet VM in your desired hypervisor from above.
 - a. VirtualBox
 - i. You can double click on the .ovf file and import it.
 - ii. If you get errors importing the .ovf file you can simply create a new VM of the appropriate type (Linux, Ubuntu 44 bit) and use the .vmdk file as the virtual hard disk for the new VM.
 - iii. You can also Open the VB and import the downloaded mininet machine.
 - iv. Select settings and add an additional host-only network adapter that you can use log into the VM image. Start the VM.
 - v. See further on VB website for further instructions, if you require.
 - b. VMWare
 - i. Import the OVF file, the start the VM
 - ii. VMware may ask you to install VMware tools on the VM - if it asks, decline. Everything graphical in the tutorial is done via X forwarding through SSH (in fact, the VM doesn't have a desktop manager installed), so the VMware tools are unnecessary unless you wish to install an X11/Gnome/etc. environment in your VM.

Step-by-step:

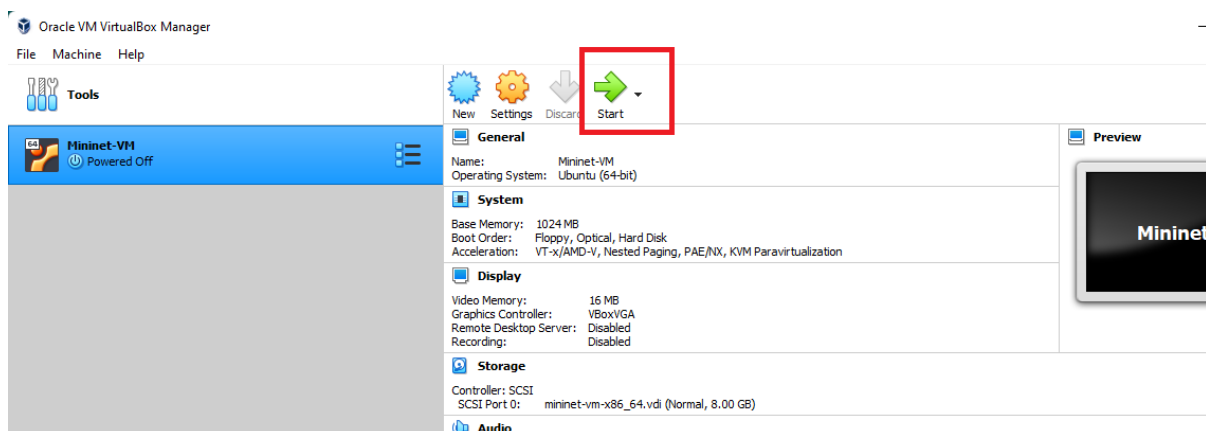
When you unzip the downloaded Mininet VM file and extract it. You will find these two files in the folder.

mininet-2.3.0-210211-ubuntu-20.04.1-legacy-serve...				Search mininet-2.3.0-210211-ubuntu-20.04.1-legacy-se	
Name	Date modified	Type	Size		
 mininet-2.3.0-210211-ubuntu-20.04.1-legacy-server-amd64	15/09/2022 17:05	Open Virtualizatio...	4 KB		
 mininet-vm-x86_64	15/09/2022 17:05	VMware virtual dis...	3,103,040 KB		

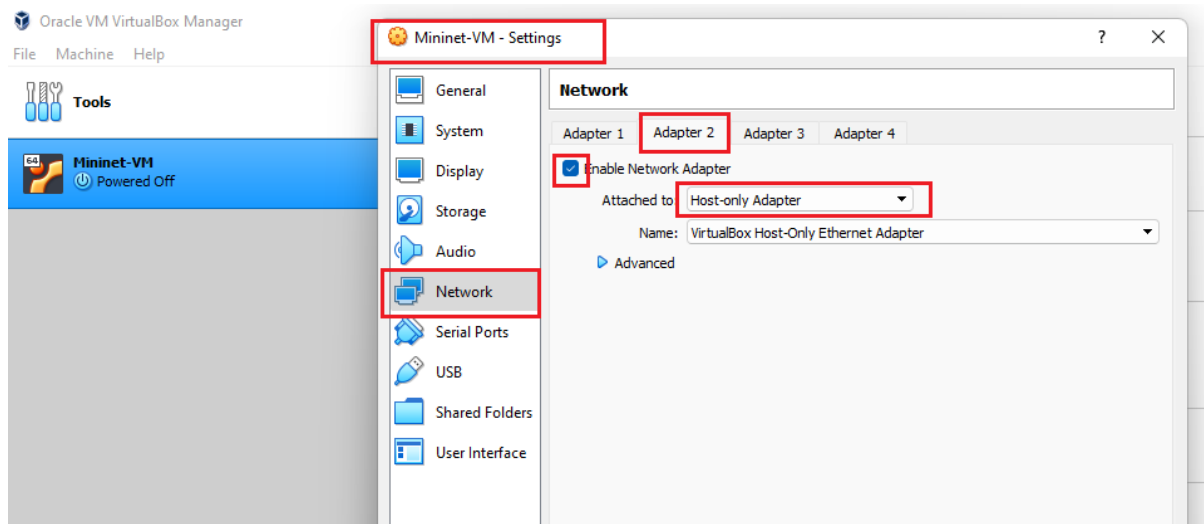
Open Virtual Box. Go in the File menu and select Import Appliances.



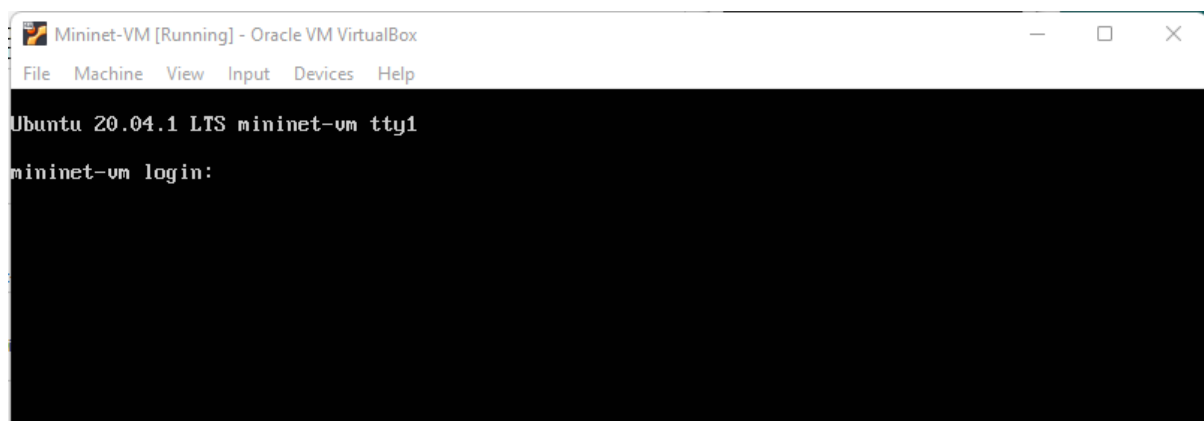
Provide the path for Ovf file and it will import the mininet VM machine.



Click on settings to add additional Network adapter as Host only. This can be used for SSH connectivity to the main Mininet machine.



Press OK, and then Start VM.

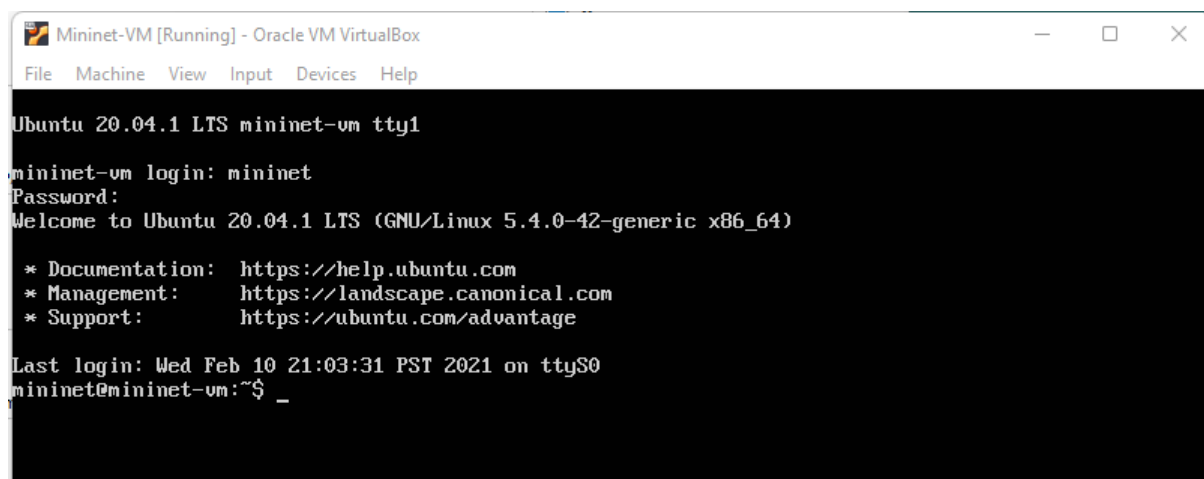


Log in to VM

Log in to the VM, using the following name and password:

mininet-vm login: mininet

Password: mininet



You can directly use the Mininet from CLI or you can SSH into Mininet VM from your host machine. Its optional, for now. You can leave SSH for now.

SSH into VM

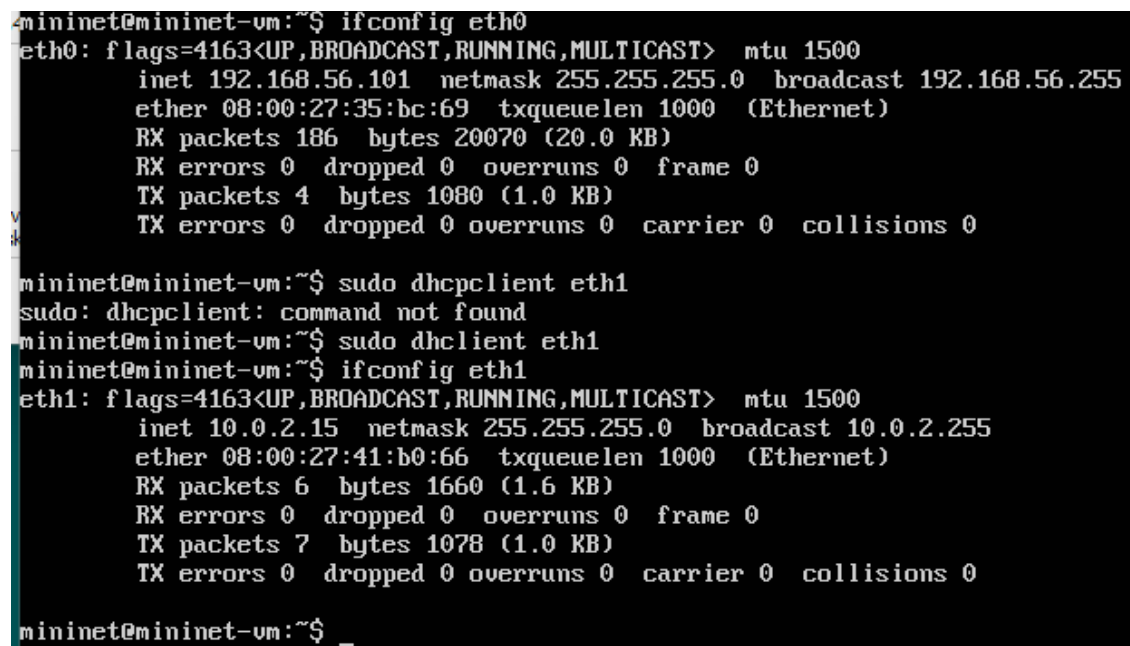
First, find the VM's IP address, which for VMware is probably in the range 192.168.x.y. In the VM console:

```
ifconfig eth0
```

Note: VirtualBox users who have set up a host-only network on eth1 should use

```
sudo dhclient eth1 # make sure that eth1 has an IP address
```

```
ifconfig eth1
```



```
mininet@mininet-vm:~$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.56.101 netmask 255.255.255.0 broadcast 192.168.56.255
    ether 08:00:27:35:bc:69 txqueuelen 1000 (Ethernet)
    RX packets 186 bytes 20070 (20.0 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4 bytes 1080 (1.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet@mininet-vm:~$ sudo dhclient eth1
sudo: dhclient: command not found
mininet@mininet-vm:~$ sudo dhclient eth1
mininet@mininet-vm:~$ ifconfig eth1
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.15 netmask 255.255.255.0 broadcast 10.0.2.255
    ether 08:00:27:41:b0:66 txqueuelen 1000 (Ethernet)
    RX packets 6 bytes 1660 (1.6 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 7 bytes 1078 (1.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet@mininet-vm:~$ _
```

SSH connectivity, if needed

You may want to add the address to your host PC's /etc/hosts file to be able to SSH in by name, if it's Unix-like. For example, add a line like this for OS X:

```
192.168.x.y mininet-vm
```

where 192.168.x.y is replaced by the VM's IP address.

SSH into the VM. We assume the VM is running locally, and that the additional precautions of `ssh -X` are unnecessary. `ssh -Y` also has no authentication timeout by default.

```
ssh -Y mininet@mininet-vm
```

If you are running the VM under QEMU/KVM with `-net user` and the `hostfwd` option as recommended above, the VM IP address is irrelevant. Instead, you tell SSH to connect to port 8022 on the host:

```
ssh -Y -p 8022 mininet@localhost
```


Basic commands:

Type sudo mn

This command can be used to start the mininet.

```

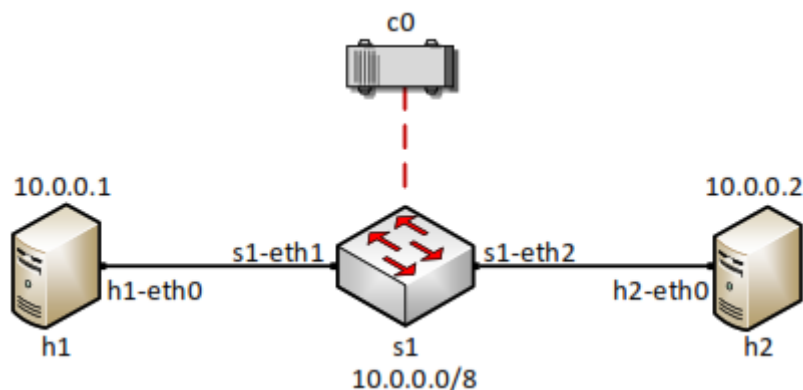
bin  dev  home  lib32  libx32  media  opt  root  sbin  swapfile  tmp  var
boot etc  lib   lib64  lost+found  mnt  proc  run  srv  sys  usr
mininet@mininet-vm:/$ cd ..
mininet@mininet-vm:/$ ls
bin  dev  home  lib32  libx32  media  opt  root  sbin  swapfile  tmp  var
boot etc  lib   lib64  lost+found  mnt  proc  run  srv  sys  usr
mininet@mininet-vm:/$ cd home/
mininet@mininet-vm:/home$ cd mininet/
mininet@mininet-vm:~$ ls
mininet  oflops  oftest  openflow  pox
mininet@mininet-vm:~$ sudo mn
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet>

```

Going into the mininet console loads a default topology with the following

- Two hosts (h1, h2)
- One Switch (s1)
- One controller (C1)
- Two links between h1 and s1, h2 and s1

The above command starts Mininet with a minimal topology, which consists of a switch connected to two hosts as shown below. The loaded topology matches the figure below.



To display the list of Mininet CLI commands and examples on their usage, type the following command.

Help

```
mininet> help

Documented commands (type help <topic>):
=====
EOF      gterm  iperfudp  nodes      pingpair    py      switch  xterm
dpctl    help   link      noecho     pingpairfull  quit    time
dump     intfs  links     pingall    ports       sh      wait
exit     iperf  net       pingallfull  px          source  x

You may also send a command to a node using:
  <node> command {args}
For example:
  mininet> h1 ifconfig

The interpreter automatically substitutes IP addresses
for node names when a node is the first arg, so commands
like
  mininet> h2 ping h3
should work.

Some character-oriented interactive commands require
noecho:
  mininet> noecho h2 vi foo.py
However, starting up an xterm/gterm is generally better:
  mininet> xterm h2

mininet> _
```

To display the available nodes, type the following command.

Nodes

```
mininet> nodes
available nodes are:
c0 h1 h2 s1
mininet>
```

The output of this command shows that there is a controller, two hosts (host h1 and host h2), and a switch (s1).

It is useful sometimes to display the links between the devices in Mininet to understand the topology. Issue the command shown below to see the available links.

Net

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0
c0
mininet>
```

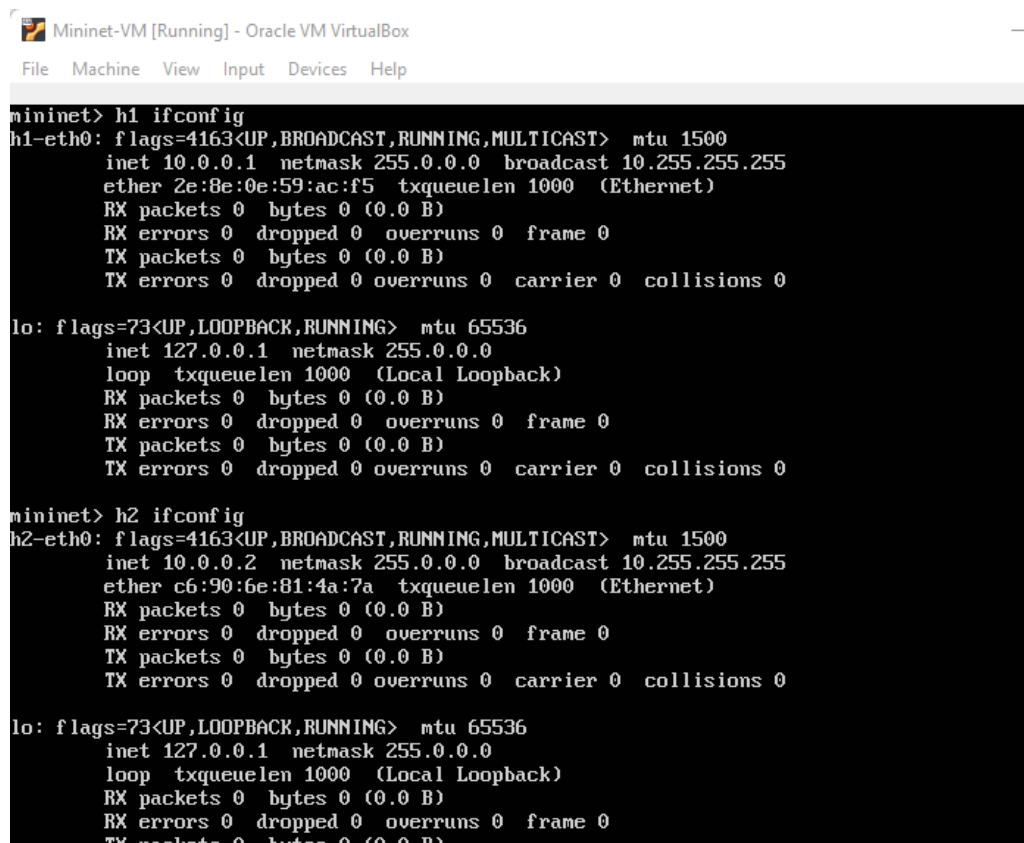
The output of this command shows that:

1. Host h1 is connected using its network interface h1-eth0 to the switch on interface s1-eth1.
2. Host h2 is connected using its network interface h2-eth0 to the switch on interface s1-eth2.
3. Switch s1:
 - a. has a loopback interface lo.
 - b. connects to h1-eth0 through interface s1-eth1.
 - c. connects to h2-eth0 through interface s1-eth2.
4. Controller c0 is the brain of the network, where it has a global knowledge about the network. A controller instructs the switches on how to forward/drop packets in the network.

Mininet allows you to execute commands on a specific device. To issue a command for a specific node, you must specify the device first, followed by the command.

To proceed, issue the following command.

h1 ifconfig



```
Mininet-VM [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 2e:8e:0e:59:ac:f5 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h2 ifconfig
h2-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.2 netmask 255.0.0.0 broadcast 10.255.255.255
    ether c6:90:6e:81:4a:7a txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
```

This command executes the `ifconfig` Linux command on host `h1`. The command shows host `h1`'s interfaces. The display indicates that host `h1` has an interface `h1-eth0` configured with IP address `10.0.0.1`, and another interface `lo`, configured with IP address `127.0.0.1` (loopback interface).

Test connectivity

Mininet's default topology assigns the IP addresses `10.0.0.1/8` and `10.0.0.2/8` to host `h1` and host `h2` respectively. To test connectivity between them, you can use the command `ping`. The `ping` command operates by sending Internet Control Message Protocol (ICMP) Echo Request messages to the remote computer and waiting for a response or reply. Information available includes how many responses are returned and how long it takes for them to return.

On the CLI, type the command shown below. This command tests the connectivity between host `h1` and host `h2`.

```

mininet> h1 ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=7.89 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.513 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.084 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.086 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.105 ms
64 bytes from 10.0.0.2: icmp_seq=6 ttl=64 time=0.123 ms
^C
--- 10.0.0.2 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5085ms
rtt min/avg/max/mdev = 0.084/1.466/7.888/2.875 ms
mininet>

```

To stop the test, press Ctrl+c. The figure above shows a successful connectivity test. Host h1 (10.0.0.1) sent four packets to host h2 (10.0.0.2) and successfully received the expected responses.

Other format of command is - **h1 ping h2**

Other Useful Commands:

xterm h1 - to run commands within a host or switch

py h1.IP() - to display IP of a host

Exit and clean-up:

Stop the emulation by typing the following command.

Exit

```

mininet> exit
**** Stopping 1 controllers
c0
**** Stopping 2 links
..
**** Stopping 1 switches
s1
**** Stopping 2 hosts
h1 h2
**** Done
completed in 763.602 seconds
mininet@mininet-vm:~$ _

```

The command **sudo mn -c** is often used on the Linux terminal (not on the Mininet CLI) to clean a previous instance of Mininet (e.g., after a crash).

Explore the files in the mininet folder to see where exactly each required file is. Also, explore the example folder to see the python files for different applications. There you can observe

how the links, switches, controllers, and hosts are implemented. We will discuss those things in the lecture and next labs.

There is also a GUI available for Mininet named as Miniedit. The GUI is not our focus. We will try to cover it to some extent in our labs. If any one is interested. Its python file is included in the example folder of mininet.

Mininet folder:

```
mininet@mininet-vm:~$ ls
mininet  oflops  oftest  openflow  pox
mininet@mininet-vm:~$ cd mininet/
mininet@mininet-vm:~/mininet$ ls
bin          custom  doc      INSTALL  Makefile  mn.1     mnexec.1  README.md  util
CONTRIBUTORS  debian  examples LICENSE  mininet   mnexec    mnexec.c  setup.py
mininet@mininet-vm:~/mininet$ cd examples
mininet@mininet-vm:~/mininet/examples$ ks
ks: command not found
mininet@mininet-vm:~/mininet/examples$ ls
baresshd.py      controllers2.py  limit.py          multitest.py     scratchnet.py
bind.py           controllers.py   linearbandwidth.py natnet.py         scratchnetuser.py
clustercli.py     controlnet.py   linuxrouter.py   nat.py           simpleperf.py
clusterdemo.py    cpu.py          miniedit.py       numberedports.py sshd.py
clusterperf.py    emptynet.py     mobility.py       popenpoll.py     test
cluster.py        hwintf.py       multilink.py      popen.py         tree1024.py
clusterSanity.py  __init__.py     multiping.py      __pycache__      treeping64.py
consoles.py       intfoptions.py  multipoll.py      README.md        vlanhost.py
mininet@mininet-vm:~/mininet/examples$ _
```

TASK:

**** No submission is required for this task**

Run another custom topology from mininet folder named as topo-2sw-2host.py

See the python file by typing vi and the file name as

Sudo vi ~/mininet/custom/topo-2sw-2host.py

****before that do sudo mn -c to clear any previous topologies**

Command:

mininet@mininet-vm:~\$ sudo mn --custom ~/mininet/custom/multi-hosts.py --topo=mytopo

```
mininet@mininet-vm:~$ sudo mn --custom ~/mininet/custom/topo-2sw-2host.py --topo=mytopo
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s3 s4
*** Adding links:
(h1, s3) (s3, s4) (s4, h2)
*** Configuring hosts
h1 h2
*** Starting controller
c0
*** Starting 2 switches
s3 s4 ...
*** Starting CLI:
mininet>
```

Now, use above commands in the lab to check the links, nodes, net, and connectivity.

```
c0 h1 h2 s3 s4
mininet> ifconfig h1
*** Unknown command: ifconfig h1
mininet> h1 ifconfig
h1-eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.0.1 netmask 255.0.0.0 broadcast 10.255.255.255
    ether 6a:bf:ea:49:79:b7 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

mininet> h1 ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=15.5 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.703 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.126 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.104 ms
^C
--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3058ms
rtt min/avg/max/mdev = 0.104/4.095/15.450/6.559 ms
mininet> _
```

Reading:

- Introduction to Mininet: <https://github.com/mininet/mininet/wiki/Introduction-to-Mininet>
- Mininet Walkthrough: <http://mininet.org/walkthrough/>
- Mininet Python API Reference Manual: <http://mininet.org/api/annotated.html>